

Lời giải trận thứ ba HDU Multi-University 2021

Nguồn gốc: HDU Multi-University 2021 MINIEYE Cup, Contest 3 Solutions

Contest / HDU 2021 MINIEYE Cup Contest 3 solutions.pdf

1 1001 Bookshop

Shortest judge solution: 3945 Bytes.

Phân rã cây nặng nhẹ và tính thứ tự DFS. Trong DFS, trước hết đi vào con nặng của một đỉnh, rồi mới đi vào các con nhẹ; khi đó mỗi chuỗi nặng là một đoạn liên tiếp trên thứ tự DFS. Với mỗi truy vấn (x, y, z) , gọi t là LCA của x và y . Quá trình đi từ x lên t tương ứng với $O(\log n)$ đoạn trên thứ tự DFS theo chiều từ phải sang trái; quá trình đi từ t xuống y cũng tương ứng với $O(\log n)$ đoạn, nhưng theo chiều từ trái sang phải. Xử lý các truy vấn offline, ta tách mỗi truy vấn thành hai bước thống nhất:

- Với mỗi truy vấn, xử lý đoạn đi từ x lên LCA, tức tách thành $O(\log n)$ đoạn theo chiều từ phải sang trái.
- Với mỗi truy vấn, xử lý đoạn đi từ LCA xuống y , tức tách thành $O(\log n)$ đoạn theo chiều từ trái sang phải.

Hai bước trên tương tự nhau. Lấy bước thứ nhất làm ví dụ. Ta duy trì một cấu trúc dữ liệu T và quét thứ tự DFS của cả cây từ phải sang trái. Giả sử đang quét đến vị trí thứ i trong thứ tự DFS; khi đó cần lần lượt xử lý các loại sự kiện sau:

- Với mỗi đoạn tách ra từ một truy vấn có đầu phải bằng i , chèn truy vấn tương ứng vào cấu trúc dữ liệu T .
- Gọi x là đỉnh do vị trí thứ i biểu diễn. Với mọi truy vấn trong T có giá trị z ít nhất là p_x , giảm giá trị z tương ứng đi p_x .
- Với mỗi đoạn tách ra từ một truy vấn có đầu trái bằng i , xóa truy vấn tương ứng khỏi cấu trúc dữ liệu T .

Vì vậy ta cần duy trì một cấu trúc dữ liệu T hỗ trợ chèn số, xóa số, và giảm mọi số không nhỏ hơn k đi k . Dùng cây cân bằng T để duy trì mọi truy vấn theo thứ tự tăng dần của z . Khi đó chèn và xóa là thao tác cơ bản. Khi cần giảm mọi giá trị z không nhỏ hơn k đi k :

- Các giá trị z trong $[0, k)$ không cần thay đổi.
- Các giá trị z trong $[k, 2k)$ cần giảm đi k ; giá trị của chúng giảm ít nhất một nửa, nên có thể sửa thô. Mỗi truy vấn tổng cộng chỉ bị sửa $O(\log z)$ lần.
- Các giá trị z trong $[2k, +\infty)$ cần giảm đi k ; thứ tự tương đối của chúng không đổi, nên có thể dùng thể lười.

Do đó tổng cộng có nhiều nhất $O(n + q \log n + q \log z)$ thao tác trên cây cân bằng. Mỗi thao tác tốn $O(\log q)$, nên tổng độ phức tạp thời gian là

$$O((n + q \log n + q \log z) \log q).$$

2 1002 Destinations

Shortest judge solution: 1910 Bytes.

Mỗi người có ba tuyến khả thi. Tuyến thứ j có điểm bắt đầu s_i , điểm kết thúc $e_{i,j}$ và chi phí $c_{i,j}$. Để thấy ba tuyến này có điểm chung là điểm bắt đầu, nên bài toán tương đương với bài toán sau: cho $3m$ đường đi trên cây, chọn m đường đi không có điểm chung sao cho tổng chi phí nhỏ nhất. Chú ý rằng nhiều nhất chỉ có thể chọn m đường đi không có điểm chung, nên bài toán lại tương đương với chọn nhiều đường đi không giao nhau nhất, và trên cơ sở đó tối thiểu hóa tổng chi phí. Với một đường đi, đặt lợi ích của nó là

$$10^6(m + 1) - c_{i,j}.$$

Trong hệ cơ số 10^6m , giá trị lợi ích này là $(1, 10^6 - c_{i,j})$, và tổng lợi ích sẽ không phát sinh nhớ. Mục tiêu là tối đa hóa tổng lợi ích. Nếu chữ số cao của tổng lợi ích trong hệ cơ số 10^6m bằng m thì có nghiệm, và tổng chi phí tương ứng bằng 10^6m trừ đi chữ số thấp của tổng lợi ích trong hệ cơ số 10^6m .

Bây giờ bài toán chuyển thành: cho một số đường đi trên cây, chọn một số đường đi đôi một không có điểm chung sao cho tổng lợi ích lớn nhất. Đây là bài toán kinh điển. Một cách giải đơn giản là xét bài toán đối ngẫu: chọn ít điểm nhất có thể trên cây, mỗi điểm được phép chọn nhiều lần, sao cho trên mỗi đường đi của cây số điểm được chọn ít nhất bằng lợi ích của đường đi đó. Bài toán đối ngẫu có thể giải tham lam: DFS cây từ sâu lên nông. Khi xét đỉnh x , xử lý mọi đường đi (u, v, w) có LCA là x , thống kê số điểm đã chọn trên đường đi từ u đến v , gọi là cnt . Nếu chưa đủ w , thì bổ sung $w - cnt$ điểm tại LCA, tức tại đỉnh x .

Vì vậy cần một cấu trúc dữ liệu hỗ trợ sửa một điểm và truy vấn tổng trên đường đi. Gọi f_x là tổng trọng số các điểm trên đường đi từ x đến gốc. Một sửa đổi đơn điểm tác động lên f dưới dạng cộng cả cây con, có thể duy trì sai phân bằng cây Fenwick.

Độ phức tạp thời gian là $O((n + m) \log n)$.

3 1003 Forgiving Matching

Shortest judge solution: 2114 Bytes.

Với mỗi chuỗi con độ dài m của S , thống kê số vị trí khớp với T , ký hiệu là f_i . Giá trị này cho ta giá trị k nhỏ nhất để chuỗi con đó được xem là khớp. Hai ký tự khớp khi và chỉ khi chúng bằng nhau, hoặc ít nhất một trong hai ký tự là ký tự đại diện. Trước hết giả sử không có ký tự đại diện. Liệt kê từng ký tự c từ 0 đến 9; nếu $S_i = T_j = c$ thì f_{i-j} cần tăng thêm 1. Đảo chuỗi T rồi dùng FFT để tính f .

Bây giờ xét ảnh hưởng của ký tự đại diện. Gọi w_S là số ký tự đại diện trong chuỗi con tương ứng của S , w_T là số ký tự đại diện trong T , và w_{ST} là số vị trí mà cả hai bên đều là ký tự đại diện. Số vị trí khớp nhờ ký tự đại diện bằng

$$w_S + w_T - w_{ST}.$$

Giá trị w_S có thể tính bằng tổng tiền tố; giá trị w_{ST} cũng có thể tính bằng FFT.

Độ phức tạp thời gian là $O(11n \log n)$.

4 1004 Game on Plane

Shortest judge solution: 872 Bytes.

Hai đường thẳng có điểm chung khi và chỉ khi chúng trùng nhau hoặc có hệ số góc khác nhau. Vì thế chiến lược tối ưu của Bob chắc chắn là tránh những đường thẳng thuộc nhóm hệ số góc xuất hiện nhiều nhất. Để khiến Bob giao với càng nhiều đường thẳng càng tốt, chiến lược tối ưu của Alice là tối thiểu hóa số lần xuất hiện lớn nhất của một hệ số góc; do đó chỉ cần liên tục chọn một đường từ mỗi loại hệ số góc.

Độ phức tạp thời gian là $O(n \log n)$.

5 1005 Kart Race

Shortest judge solution: 2612 Bytes.

Bắt đầu DFS toàn bộ đồ thị từ đỉnh 1 và ghi lại thứ tự ra khỏi ngăn xếp. Nếu x đi được đến y , hiển nhiên x ra khỏi ngăn xếp muộn hơn y . Vì đồ thị là đồ thị phẳng, xét hai cách duyệt tiêu biểu nhất: duyệt theo chiều kim đồng hồ và duyệt ngược chiều kim đồng hồ. Như vậy ta thu được hai thứ tự ra khỏi ngăn xếp. Gọi a_i là thứ tự ra khỏi ngăn xếp của đỉnh i khi duyệt đồ thị theo chiều kim đồng hồ, và b_i là thứ tự tương ứng khi duyệt ngược chiều kim đồng hồ. Khi đó x đi được đến y khi và chỉ khi

$$a_x > a_y \quad \text{và} \quad b_x > b_y.$$

Theo đề bài, các điểm được chọn phải đôi một không đi được đến nhau. Do đó, nếu xem a_i là hoành độ và b_i là tung độ, thì khi chọn (a_i, b_i) ta không thể chọn mọi điểm ở góc dưới trái và góc trên phải của nó. Vì vậy dãy điểm được chọn chắc chắn thỏa a tăng dần từ trái sang phải và b giảm dần. Bài toán chuyển thành tìm dãy con giảm có tổng giá trị lớn nhất; dùng cây Fenwick để tối ưu DP thô và thu được lời giải trong $O(n \log n)$.

Để tìm nghiệm tối ưu nhỏ nhất theo thứ tự từ điển, một cách là ghi trực tiếp trong giá trị DP một chuỗi nhị phân độ dài n cho biết phương án hiện tại đã chọn những điểm nào. Khi chuyển trạng thái cần hỗ trợ so sánh thứ tự từ điển, sao chép, và sửa một vị trí từ 0 thành 1. Vì vậy dùng cây đoạn bền vững để ghi phương án. Sửa một điểm tốn $O(\log n)$. Khi so sánh thứ tự từ điển của hai phương án, dựa vào việc cây con trái có giống nhau hay không để quyết định đệ quy sang trái hay sang phải; độ phức tạp cũng là $O(\log n)$. Ở đây, chú ý rằng mỗi điểm chỉ được thêm một lần, nên nếu con trỏ gốc của hai cây đoạn khác nhau thì hai chuỗi nhị phân được biểu diễn chắc chắn khác nhau; không cần duy trì thêm giá trị băm.

Độ phức tạp thời gian là $O(n \log^2 n)$.

6 1006 New Equipments II

Shortest judge solution: 1711 Bytes.

Xây đồ thị luồng mạng như sau:

- Bên trái có n đỉnh biểu diễn n công nhân, bên phải có n đỉnh biểu diễn n thiết bị; thêm nguồn S và đích T .
- Nối cạnh từ S đến công nhân thứ i , dung lượng 1, chi phí a_i .
- Nối cạnh từ thiết bị thứ i đến T , dung lượng 1, chi phí b_i .
- Nếu công nhân i có thể ghép với thiết bị j , nối cạnh từ công nhân i đến thiết bị j , dung lượng 1, chi phí 0.

Mục tiêu là tìm luồng chi phí lớn nhất với lưu lượng lần lượt bằng $1, 2, 3, \dots, n$. Ta tăng luồng n lần; mỗi lần cần tìm hiệu quả một đường tăng luồng rồi tăng theo đường đó. Vì độ dài đường tăng luồng là $O(n)$, nút thắt của thuật toán nằm ở việc tìm đường tăng luồng.

Trong mỗi lần tăng luồng, ta cần tìm một đường từ S đến T có chi phí lớn nhất. Chú ý rằng chi phí khác 0 chỉ xuất hiện trên các cạnh nối mỗi đỉnh với nguồn hoặc đích. Vì vậy điều này tương đương với tìm một công nhân x và một thiết bị y sao cho chúng chưa được dùng trong các lần tăng luồng trước, x có thể đi đến y thông qua các cạnh ban đầu và $O(n)$ cạnh ghép được thêm sau các lần tăng luồng để cho phép đổi ý, đồng thời $a_x + b_y$ lớn nhất. Liệt kê mỗi đỉnh y ở phía phải; chỉ cần tìm đỉnh phía trái đi được đến nó có trọng số lớn nhất là ta tìm được cặp đỉnh tối ưu và đường tăng luồng tương ứng.

Sắp xếp các đỉnh phía trái theo a giảm dần rồi lần lượt bắt đầu BFS từ mỗi đỉnh. Ghi lại mỗi đỉnh lần đầu được đỉnh phía trái nào duyệt tới; đỉnh đó chính là đỉnh phía trái có trọng số lớn nhất có thể đi đến nó. Vì đồ thị ở đây được cho dưới dạng đồ thị bù, cần dùng BFS trên đồ thị bù: duy trì tập R gồm các đỉnh chưa được duyệt; khi duyệt đến đỉnh x , sửa R thành tập các đỉnh kề ra của x trong đồ thị bù. Vì đồ thị bù chỉ có m cạnh, cách BFS này có độ phức tạp $O(n + m)$.

Cần tăng luồng tổng cộng n vòng. Mỗi vòng tìm đường tăng luồng trong $O(n + m)$, nên tổng độ phức tạp thời gian là

$$O(n^2 + nm).$$

7 1007 Photoshop Layers

Shortest judge solution: 674 Bytes.

Tiền xử lý f_i là lớp đầu tiên ở bên trái lớp i có chế độ hợp thành “normal”. Với mỗi truy vấn, tìm lớp đầu tiên ở bên trái r có chế độ hợp thành “normal”, tức f_r . Phần ở giữa đều là “linear dodge”, nên có thể dùng tổng tiền tố để tính kết quả, rồi cuối cùng lấy giá trị nhỏ hơn với 255.

Độ phức tạp thời gian là $O(n + q)$.

8 1008 Restore Atlantis II

Shortest judge solution: 4143 Bytes.

Xét trường hợp một chiều. Với dữ liệu ngẫu nhiên, hợp của n đoạn thẳng có thể được biểu diễn bằng rất ít đoạn cực dài; trong hai chiều tính chất này cũng đúng. Một cách cài đặt đơn giản là trích xuất mọi điểm mốc theo phương x . Với mỗi khoảng giữa hai điểm mốc liên tiếp, dùng `std::vector` để ghi hợp các đoạn theo phương y được tạo bởi những hình chữ nhật cực dài nào. Sau đó lần lượt xét các phần nằm giữa hai đoạn mốc kề nhau theo phương x ; nếu các `std::vector` mà chúng ghi lại giống nhau thì gộp hai đoạn đó thành cùng một đoạn, qua đó đạt hiệu quả nén. Nhờ vậy, với dữ liệu ngẫu nhiên, hợp hình chữ nhật của bất kỳ khoảng nào cũng có thể được ghi bằng khoảng vài chục đơn vị thông tin.

Phần còn lại là truy vấn hợp hình chữ nhật trên một đoạn chỉ số. Nếu dùng cây đoạn để duy trì hợp hình chữ nhật trên đoạn, tiền xử lý cần $O(n)$ lần gộp thông tin, và mỗi truy vấn cần $O(\log n)$ lần gộp thông tin, hiệu quả còn thấp. Ta chia n hình chữ nhật thành $O(n/\log n)$ khối, mỗi khối có $O(\log n)$ hình chữ nhật. Tiền xử lý hợp tiền tố và hậu tố bên trong mỗi khối, đồng thời dùng bảng thưa để tiền xử lý hợp hình chữ nhật giữa hai khối bất kỳ. Khi đó tiền xử lý cần $O(n)$ lần gộp thông tin. Với mỗi truy vấn $[l, r]$:

- Nếu l và r nằm trong hai khối khác nhau, thông tin cuối cùng bằng hợp của phần hậu tố trong khối chứa l , phần tiền tố trong khối chứa r , và toàn bộ các khối nằm giữa; chỉ cần $O(1)$ lần gộp thông tin.

- Nếu l và r nằm trong cùng một khối, do dữ liệu ngẫu nhiên, số truy vấn như vậy có kỳ vọng $O(\log n)$; có thể tính thô hợp thông tin.

Cuối cùng tổng cộng cần $O(n + q)$ lần gộp thông tin. Tốc độ chạy thực tế phụ thuộc khá nhiều vào cách cài đặt thao tác gộp thông tin.

9 1009 Rise in Price

Shortest judge solution: 1326 Bytes.

Đặt $f_{i,j,k}$ là giá cao nhất mà đơn giá kim cương có thể tăng tới khi đi từ $(1, 1)$ đến (i, j) và trên đường đi đã thu thập k viên kim cương. Khi đó

$$ans = \max_k (k \cdot f_{n,n,k}).$$

Với một ô cố định (i, j) , xét hai trạng thái $f_{i,j,x}$ và $f_{i,j,y}$, trong đó $x < y$. Nếu $f_{i,j,x} \leq f_{i,j,y}$ thì trạng thái $f_{i,j,x}$ chắc chắn không thể phát triển thành nghiệm tối ưu, nên có thể loại bỏ. Với mỗi ô (i, j) , dùng một danh sách lưu mọi trạng thái theo thứ tự tăng dần của k , đồng thời loại bỏ những trạng thái không thể trở thành nghiệm tối ưu.

Với dữ liệu ngẫu nhiên và $n = 100$, giá trị đỉnh của số trạng thái hữu hiệu ứng với một ô (i, j) xấp xỉ vài nghìn. Độ phức tạp thời gian là $O(n^2k)$.

10 1010 Road Discount

Shortest judge solution: 1425 Bytes.

Xem cạnh gốc là cạnh trắng, cạnh giảm giá là cạnh đen. Vì cùng một cạnh không thể được chọn hai lần, bài toán tương đương với tìm cây khung nhỏ nhất chứa đúng k cạnh đen. Đây là bài toán kinh điển. Gọi $f(k)$ là tổng trọng số của cây khung nhỏ nhất chứa đúng k cạnh đen; khi đó $f(k)$ là một hàm lồi. Cách tính $f(k)$ như sau:

- Chọn tham số c và cộng c vào trọng số của mỗi cạnh đen.
- Tìm cây khung nhỏ nhất của đồ thị sau khi sửa trọng số. Gọi $sum(c)$ là tổng trọng số tương ứng, $l(c)$ là số cạnh đen nhỏ nhất có thể xuất hiện trong một cây khung nhỏ nhất, và $r(c)$ là số cạnh đen lớn nhất có thể xuất hiện trong một cây khung nhỏ nhất.
- Nhị phân để tìm tham số c thích hợp sao cho $l(c) \leq k \leq r(c)$. Khi đó

$$f(k) = sum(c) - k \cdot c.$$

Do trọng số cạnh nằm trong $[1, 1000]$, có thể tiền xử lý toàn bộ thông tin cho $c = -1000, \dots, 1000$, tức cần tính $O(c)$ lần cây khung nhỏ nhất. Chú ý rằng nếu tìm riêng cây khung nhỏ nhất chỉ trên cạnh đen hoặc chỉ trên cạnh trắng, thì các cạnh không thuộc cây sẽ không bao giờ được dùng; vì vậy có thể giảm số cạnh xuống $O(n)$.

Tổng độ phức tạp thời gian là $O(m \log n + nc \log n)$.

11 1011 Segment Tree with Pruning

Shortest judge solution: 371 Bytes.

Trên cây đoạn, các nút biểu diễn những đoạn có cùng độ dài thì số đỉnh trong cây con cũng giống nhau. Hơn nữa chỉ có nhiều nhất $O(\log n)$ độ dài đoạn khác nhau về bản chất. Vì vậy chỉ cần DFS có nhớ theo độ dài đoạn.

Độ phức tạp thời gian là $O(\log n \log \log n)$.

12 1012 Tree Planting

Shortest judge solution: 2850 Bytes.

Thuật toán 1: đặt $f_{i,S}$ là tổng đóng góp sau khi đã xét i vị trí đầu, trong đó tình trạng trồng cây của k vị trí trước i là S . Độ phức tạp thời gian là $O(n2^k)$.

Thuật toán 2: đặt vị trí i vào ô

$$\left(i \bmod k, \left\lfloor \frac{i}{k} \right\rfloor \right)$$

của một ma trận hai chiều. Ma trận này có k hàng và $\left\lceil \frac{n}{k} \right\rceil$ cột. Nếu ô (i, j) được trồng cây, thì các ô $(i-1, j)$, $(i+1, j)$, $(i, j-1)$ và $(i, j+1)$ đều không thể trồng cây. Ngoài ra hàng đầu và hàng cuối cũng có xung đột. Làm DP đường viền từ trên xuống dưới, từ trái sang phải. Đặt $f_{i,j,S,T}$ là tổng đóng góp khi đã xét đến ô (i, j) , trong đó tình trạng trồng cây của hàng đầu là S và tình trạng trên đường viền là T . Độ phức tạp thời gian là

$$O(n2^{2n/k}).$$

Chú ý rằng trong cả hai thuật toán, S và T chắc chắn là tập độc lập. Số tập độc lập trên một đường đi gồm k đỉnh là số Fibonacci thứ k , xấp xỉ $O(1.618^k)$. Vì vậy thuật toán 1 giảm xuống

$$O(n1.618^k),$$

còn thuật toán 2 giảm xuống

$$O(n1.618^{2n/k}).$$

Khi $k \leq 2n/k$, tức $k \leq \sqrt{2n}$, dùng thuật toán 1; khi $k > \sqrt{2n}$, dùng thuật toán 2. Ta thu được độ phức tạp tối ưu

$$O(n1.618^{\sqrt{2n}}).$$